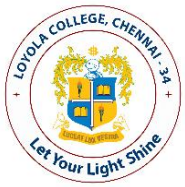
LOYOLA COLLEGE

Chennai – 600 034



Department of Data Science

PRACTICAL RECORD

Course Title : Python For Data Science

Dept. No. : 26-PDS-014

Name : MARIYA JOHNET R

Semester : I

Year : 2026-2027

Loyola College, Chennai – 600 034

Department of Data Science

BONAFIDE CERTIFICATE

Dept. No. : 26-PDS-014

Name : MARIYA JOHNET R

Class : 1 M.Sc Data Science

This is to certify that this is the bonafide record of the practical work done in Python for Data Science Lab at the Loyola College Data Center, during the year 2026-2027.

Head of the Department **Staff-in-Charge**

Date : 13.10.2026

Submitted for the Python for Data Science Practical examination held on 13.10.2026 at Loyola College, Chennai – 600 034.

Internal Examiner **External Examiner**

INDEX

S.No.	Date	Name of the Exercise	Page No.
1	29-06-2026	Flow Control	4
2	02-07-2026	Functions	7
3	07-07-2026	String Manipulation	10
4	10-07-2026	Creating and Manipulating Tuple	13
5	13-07-2026	Creating and Manipulating a List	16
6	17-07-2026	Creating and Manipulating a Dictionary	20
7	24-07-2026	Object Creation and Usage	23
8	28-07-2026	Program Involving Inheritance	26
9	06-08-2026	Program Involving Overloading	29
10	20-08-2026	Reading and Writing Text and Binary Files	32
11	28-08-2026	Combining and Merging Datasets	35
12	07-09-2026	Program Involving Regular Expressions	38
13	22-09-2026	Data Aggregation and Group-wise Operations	41

Experiment No. 1

Title

Editing and executing programs involving flow control.

Aim

To write a Python program that uses flow control statements (for loops and conditional statements) to compute the mean, standard deviation, and z-scores of a dataset, and to identify outliers based on a z-score threshold.

Theory

Flow control statements allow a program to make decisions and repeat operations. The two main constructs used in this program are:

- **Iteration (for loop):** Used to traverse a list of values repeatedly and accumulate results, such as sums.
- **Selection (if statement):** Used to test a condition and execute code only when that condition is true.

Algorithm

1.Start.

2.Initialize the list data with the given values.
3.Set n to the number of elements in data.
4.Step to compute mean:
 a. Initialize total = 0.
 b. For each value in data, add it to total.
 c. Compute mean = total / n.
5.Step to compute standard deviation:
 a. Initialize sq_diff_sum = 0.
 b. For each value in data, compute (value − mean)² and add it to sq_diff_sum.
 c. Compute std_dev = (sq_diff_sum / n) ** 0.5.
6.Set threshold k = 2 and initialize an empty list outliers.
7.Step to detect outliers:
 a. For each value in data, compute z_score = (value − mean) / std_dev.
 b. If abs(z_score) > k, append (value, z_score) to outliers.
8.Print the dataset, mean, and standard deviation.

9.Print each outlier along with its z-score.

10.Print the total number of records, number of outliers, and percentage of outliers.

11.Stop.

Program

```
data = [12, 15, 14, 10, 13, 200, 11, 9, 16, 14, -50, 12, 15]
n = len(data)
total = 0
for value in data:
  total += value
mean = total / n
sq_diff_sum = 0
for value in data:
  sq_diff_sum += (value - mean) ** 2
std_dev = (sq_diff_sum / n) ** 0.5
k = 2
outliers = []
for value in data:
  z_score = (value - mean) / std_dev
  if abs(z_score) > k:
    outliers.append((value, round(z_score, 2)))
print(f"Dataset: {data}")
print(f"Mean: {mean:.2f}, Std Dev: {std_dev:.2f}")
print(f"Outliers (|z| > {k}):")
for val, z in outliers:
  print(f" Value = {val}, z-score = {z}")
print(f"Total records: {n}, Outliers found: {len(outliers)} "
  f"({(len(outliers)/n)*100:.1f}% of data)")
```

Expected Output

```
Dataset: [12, 15, 14, 10, 13, 200, 11, 9, 16, 14, -50, 12, 15]
Mean: 22.38, Std Dev: 53.95
Outliers (|z| > 2):
 Value = 200, z-score = 3.29
Total records: 13, Outliers found: 1 (7.7% of data)
```

Observation

- Program executed successfully with no errors.
- Correct output obtained: Mean = 22.38, Std Dev = 53.95.
- Only 200 (z = 3.29) was flagged as an outlier out of 13 records.

Result

The objective of the experiment was achieved successfully. The program correctly used flow control (loops and conditionals) to compute mean, standard deviation, and detect outliers using the z-score method.

Learning Outcome

- Learned to use for loops for cumulative calculations and if statements for conditional filtering.
- Understood how to compute mean, standard deviation, and z-score in Python.
- Learned that extreme values can distort the mean/std dev, affecting outlier detection.

Experiment No. 2

Title

Editing and executing programs involving functions.

Aim

To write a Python program that demonstrates user-defined functions using default arguments, variable-length arguments (*args), keyword arguments (**kwargs), return values, and recursion.

Theory

A function is a reusable block of code defined using the def keyword that performs a specific task when called. Python supports several flexible ways of passing arguments to functions:

- Default Arguments — a parameter can be assigned a default value (e.g., greeting="Hello"), which is used automatically if the caller does not supply that argument.
- *args (Variable Positional Arguments) — allows a function to accept any number of positional arguments, which are collected into a tuple inside the function.
- **kwargs (Variable Keyword Arguments) — allows a function to accept any number of named (keyword) arguments, which are collected into a dictionary inside the function.
- Return Values — the return statement sends a result back to the caller, allowing the result to be stored, printed, or reused.
- Recursion — a function that calls itself to solve a smaller instance of the same problem, terminating when a base case is reached (e.g., factorial(0) or factorial(1) returning 1).
-

Algorithm

1. Start.
2. Define function greet(name, greeting="Hello") that returns a formatted greeting string using the default or supplied greeting.
3. Define function add_numbers(*args) that returns the sum of all positional arguments passed to it.
4. Define function student_info(**kwargs) that iterates through the keyword arguments and prints each key-value pair.
5. Call greet("Ravi") using the default greeting and print the result.
6. Call greet("Meena", greeting="Welcome") overriding the default and print the result.
7. Call add_numbers(2, 4, 6, 8) and print the returned sum.
8. Call student_info() with name, age, and department as keyword arguments.

9. Define recursive function factorial(n):
    a. If n == 0 or n == 1, return 1 (base case).
    b. Otherwise, return n * factorial(n - 1) (recursive case).
10. Call factorial(6) and print the result.
11. Stop.

Program

```
# Program demonstrating functions: default args, *args, **kwargs, return
values
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"
def add_numbers(*args):
    return sum(args)
def student_info(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")
# Function calls
print(greet("Ravi"))
print(greet("Meena", greeting="Welcome"))
print("Sum:", add_numbers(2, 4, 6, 8))
student_info(name="Arjun", age=20, department="CSE")
# Recursive function example
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)
print("Factorial of 6:", factorial(6))
```

Expected Output

```
Hello, Ravi!
Welcome, Meena!
Sum: 20
name: Arjun
age: 20
department: CSE
Factorial of 6: 720
```

Observation

- Program executed successfully with no errors.
- greet() used its default value "Hello" when no greeting was supplied, and correctly used "Welcome" when explicitly passed.
- add_numbers(*args) correctly summed a variable number of arguments (2 + 4 + 6 + 8 = 20).
- student_info(**kwargs) correctly accepted and printed an arbitrary number of named fields.
- factorial(6) correctly computed 720 by recursively calling itself until the base case (n == 1) was reached.

Result

The objective of the experiment was achieved successfully. The program correctly demonstrated the use of default arguments, *args, **kwargs, return values, and recursion in Python functions.

Learning Outcome

- Learned to define functions with default parameter values.
- Understood how *args allows a function to accept a variable number of positional arguments.
- Understood how **kwargs allows a function to accept a variable number of keyword arguments.
- Learned the difference between functions that return a value versus functions that print directly.
- Understood the concept of recursion, including the importance of a base case to prevent infinite function calls.

Experiment No. 3

Title

Editing and executing programs involving string manipulation.

Aim

To write a Python program that demonstrates various string manipulation operations such as stripping whitespace, case conversion, replacement, slicing, splitting, joining, string property checks, and counting substring occurrences.

Theory

Strings in Python are immutable sequences of characters and support a wide range of built-in methods for manipulation. The key operations used in this program are:

- Whitespace removal — strip() removes leading and trailing whitespace from a string without affecting the middle content.
- Case conversion — upper() and lower() convert all characters to uppercase or lowercase respectively.
- Replacement — replace(old, new) returns a new string with all occurrences of old substituted by new.
- Slicing — string[start:stop:step] extracts a substring or reverses a string (using a step of -1) without modifying the original.
- Splitting and Joining — split() breaks a string into a list of words based on whitespace, and join() combines a list of strings into one string using a specified separator.
- String property checks — methods like isalnum() and startswith() test whether a string satisfies a particular condition, returning a Boolean value.
- Counting — count(substring) returns the number of non-overlapping occurrences of a substring within a string.

Algorithm

1. Start.
2. Initialize the string text with leading and trailing spaces.
3. Print the original string using repr() to reveal whitespace.
4. Print the stripped version of the string using strip().
5. Print the string converted to uppercase using upper().
6. Print the string converted to lowercase using lower().
7. Print the string with "Fun" replaced by "Powerful" using replace().
8. Store the stripped string in clean_text.
9. Print the first 6 characters of clean_text using slicing [:6].

10. Print the reversed string using slicing [::-1].
11. Split clean_text into a list of words using split() and print it.
12. Join the list of words using - as a separator and print the result.
13. Check and print whether "Python123" is alphanumeric using isalnum().
14. Check and print whether clean_text starts with "Python" using startswith().
15. Initialize a sentence string and count the occurrences of "the" using count().
16. Print the count.
17. Stop.

Program

```
# Program demonstrating string manipulation operations
text = "  Python Programming is Fun  "
# Basic operations
print("Original:", repr(text))
print("Stripped:", text.strip())
print("Upper case:", text.upper())
print("Lower case:", text.lower())
print("Replaced:", text.replace("Fun", "Powerful"))
# Slicing
clean_text = text.strip()
print("First 6 chars:", clean_text[:6])
print("Reversed:", clean_text[::-1])
# Splitting and joining
words = clean_text.split()
print("Words:", words)
print("Joined with '-':", "-".join(words))
# Checking string properties
print("Is alphanumeric:", "Python123".isalnum())
print("Starts with 'Python':", clean_text.startswith("Python"))
# Counting occurrences
sentence = "the quick brown fox jumps over the lazy dog the end"
print("Count of 'the':", sentence.count("the"))
```

Expected Output

```
Original: '  Python Programming is Fun  '
Stripped: Python Programming is Fun
Upper case:   PYTHON PROGRAMMING IS FUN
Lower case:   python programming is fun
Replaced:   Python Programming is Powerful
First 6 chars: Python
```

```
Reversed: nuF si gnimmargorP nohtyP
Words: ['Python', 'Programming', 'is', 'Fun']
Joined with '-': Python-Programming-is-Fun
Is alphanumeric: True
Starts with 'Python': True
Count of 'the': 3
```

Observation

- Program executed successfully with no errors.
- strip() removed only the leading and trailing spaces, while upper(), lower(), and replace() preserved the original spacing since they were applied to the un-stripped text.
- Slicing correctly extracted the first 6 characters ("Python") and reversed the entire string.
- split() and join() correctly transformed the sentence into a list of words and back into a single hyphen-separated string.
- "Python123".isalnum() returned True since it contains only letters and digits, and clean_text.startswith("Python") correctly returned True.
- count("the") correctly identified 3 occurrences of "the" in the sentence, including the one inside "the end" but not overlapping with other words.

Result

The objective of the experiment was achieved successfully. The program correctly demonstrated string manipulation operations including stripping, case conversion, replacement, slicing, splitting, joining, property checking, and substring counting.

Learning Outcome

- Learned various built-in string methods such as strip(), upper(), lower(), and replace().
- Understood how slicing with [:] and [::-1] can extract substrings and reverse strings.
- Learned how split() and join() convert between strings and lists of words.
- Understood how string property methods like isalnum() and startswith() return Boolean results for condition checking.
- Learned how count() can be used to find the frequency of a substring within a larger string.

Experiment No. 4

Title

Creating and manipulating a tuple.

Aim

To write a Python program that demonstrates the creation of a tuple and basic tuple operations such as indexing, slicing, counting, concatenation, repetition, looping, and conversion to a list.

Theory

A tuple is an ordered, immutable collection of items defined using (). Elements are accessed using indexing and slicing, similar to lists. Tuples support methods like count() and index(), and can be combined using + (concatenation) or repeated using *. Since tuples cannot be modified directly, they are converted to a list when an item needs to be added.

Algorithm

1. Start.
2. Create a tuple fruits containing fruit names (including a duplicate).
3. Print the tuple.
4. Access and print the first element, last element, and a slice of the tuple.
5. Print the length using len().
6. Print the count of "banana" using count() and the index of "cherry" using index().
7. Concatenate fruits with another tuple and print the result.
8. Demonstrate repetition of a tuple using * and print the result.
9. Use a for loop to print each element of the tuple.
10. Convert fruits to a list, append a new element, convert it back to a tuple, and print the result.
11. Stop.

Program

```
# Program demonstrating tuple creation and manipulation
fruits = ("apple", "banana", "cherry", "mango", "banana")
print("Tuple:", fruits)

# Accessing elements
print("First element:", fruits[0])
print("Last element:", fruits[-1])
print("Slice [1:4]:", fruits[1:4])
```

```
# Basic properties
print("Length:", len(fruits))
print("Count of 'banana':", fruits.count("banana"))
print("Index of 'cherry':", fruits.index("cherry"))

# Concatenation and repetition
more_fruits = fruits + ("grape", "kiwi")
print("Concatenated:", more_fruits)
print("Repeated:", (1, 2) * 3)

# Looping through a tuple
print("Fruits list:")
for fruit in fruits:
    print(fruit)

# Converting tuple to list to add an item
fruit_list = list(fruits)
fruit_list.append("orange")
updated_tuple = tuple(fruit_list)
print("Updated tuple:", updated_tuple)
```

Expected Output

```
Tuple: ('apple', 'banana', 'cherry', 'mango', 'banana')
First element: apple
Last element: banana
Slice [1:4]: ('banana', 'cherry', 'mango')
Length: 5
Count of 'banana': 2
Index of 'cherry': 2
Concatenated: ('apple', 'banana', 'cherry', 'mango', 'banana', 'grape',
'kiwi')
Repeated: (1, 2, 1, 2, 1, 2)
Fruits list:
apple
banana
cherry
mango
banana
Updated tuple: ('apple', 'banana', 'cherry', 'mango', 'banana', 'orange')
```

Observation

- Program executed successfully with no errors.
- Indexing and slicing correctly retrieved individual elements and sub-tuples.
- count() and index() correctly returned 2 and 2 respectively.
- Concatenation and repetition using + and * produced new tuples as expected.
- The for loop correctly printed each fruit in the tuple.
- Converting the tuple to a list allowed a new item to be added, and converting back to a tuple produced the updated result.

Result

The objective of the experiment was achieved successfully. The program correctly demonstrated basic tuple creation and manipulation using indexing, slicing, built-in methods, concatenation, repetition, looping, and conversion to a list.

Learning Outcome

- Learned how to create and access tuple elements using indexing and slicing.
- Understood that tuples are immutable.
- Learned to use built-in methods count() and index().
- Understood tuple concatenation and repetition.
- Learned how to convert a tuple to a list when modification is needed, and back to a tuple.

Experiment No. 5

Title

Creating and manipulating a list.

Aim

To write a Python program that demonstrates the creation of a list and basic list operations such as indexing, slicing, adding, removing, modifying, sorting, reversing, and looping.

Theory

A list is an ordered, mutable collection of items defined using []. Unlike tuples, list elements can be added, removed, or changed after creation. Common list operations include append() and insert() for adding elements, remove() and pop() for deleting elements, direct index assignment for modifying elements, and sort()/reverse() for reordering. Built-in functions like len(), sum(), max(), and min() provide quick information about a list's contents.

Algorithm

1. Start.
2. Create a list numbers containing integer values.
3. Print the list.
4. Access and print the first element, last element, and a slice of the list.
5. Add an element to the end using append() and print the list.
6. Insert an element at a specific position using insert() and print the list.
7. Remove a specific element using remove() and print the list.
8. Remove the last element using pop() and print the list.
9. Modify an element by index assignment and print the list.
10. Sort the list using sort() and print it.
11. Reverse the list using reverse() and print it.
12. Print the length, sum, maximum, and minimum of the list.
13. Use a for loop to print each element of the list.
14. Stop.

Program

```
# Program demonstrating list creation and manipulation
numbers = [10, 20, 30, 40, 50]
print("List:", numbers)

# Accessing elements
```

```
print("First element:", numbers[0])
print("Last element:", numbers[-1])
print("Slice [1:4]:", numbers[1:4])

# Adding elements
numbers.append(60)
print("After append:", numbers)

numbers.insert(2, 25)
print("After insert:", numbers)

# Removing elements
numbers.remove(25)
print("After remove:", numbers)

numbers.pop()
print("After pop:", numbers)

# Modifying an element
numbers[0] = 100
print("After modification:", numbers)

# Sorting and reversing
numbers.sort()
print("Sorted:", numbers)

numbers.reverse()
print("Reversed:", numbers)

# Basic properties
print("Length:", len(numbers))
print("Sum:", sum(numbers))
print("Max:", max(numbers))
print("Min:", min(numbers))

# Looping through a list
print("List elements:")
for num in numbers:
    print(num)
```

Expected Output

```
List: [10, 20, 30, 40, 50]
First element: 10
Last element: 50
Slice [1:4]: [20, 30, 40]
After append: [10, 20, 30, 40, 50, 60]
After insert: [10, 20, 25, 30, 40, 50, 60]
After remove: [10, 20, 30, 40, 50, 60]
After pop: [10, 20, 30, 40, 50]
After modification: [100, 20, 30, 40, 50]
Sorted: [20, 30, 40, 50, 100]
Reversed: [100, 50, 40, 30, 20]
Length: 5
Sum: 240
Max: 100
Min: 20
List elements:
100
50
40
30
20
```

Observation

- Program executed successfully with no errors.
- Indexing and slicing correctly retrieved individual elements and sub-lists.
- append() and insert() correctly added elements at the end and at a specific position.
- remove() and pop() correctly deleted elements by value and by position.
- Index assignment correctly modified an existing element.
- sort() and reverse() correctly reordered the list in ascending and then descending order.
- len(), sum(), max(), and min() correctly returned the list's statistics.
- The for loop correctly printed each element of the final list.

Result

The objective of the experiment was achieved successfully. The program correctly demonstrated list creation and manipulation, including adding, removing, modifying, sorting, reversing, and looping through elements.

Learning Outcome

- Learned how to create and access list elements using indexing and slicing.
- Understood that lists are mutable and can be modified after creation.
- Learned to use list methods append(), insert(), remove(), pop(), sort(), and reverse().
- Understood the difference between a mutable list and an immutable tuple.
- Learned to use built-in functions len(), sum(), max(), and min() on a list.

Experiment No. 6

Title

Creating and manipulating a dictionary.

Aim

To write a Python program that demonstrates the creation of a dictionary and basic dictionary operations such as accessing, adding, modifying, removing key-value pairs, checking key existence, and looping.

Theory

A dictionary is an unordered, mutable collection of key-value pairs defined using {}, where each key is unique and maps to a value. Values are accessed using their key (e.g., student["name"]) or safely using get(). Key-value pairs can be added or modified by simple assignment (dict[key] = value) and removed using pop(). The in operator checks whether a key exists, and keys(), values(), and items() return the dictionary's keys, values, and key-value pairs respectively.

Algorithm

1. Start.
2. Create a dictionary student with keys "name", "age", and "department".
3. Print the dictionary.
4. Access and print the value for "name" using indexing, and "age" using get().
5. Add a new key-value pair "cgpa" and print the updated dictionary.
6. Modify the value of "age" and print the updated dictionary.
7. Remove the "cgpa" key using pop() and print the updated dictionary.
8. Check whether "name" and "marks" exist as keys using the in operator and print the results.
9. Print all keys, values, and items of the dictionary.
10. Use a for loop with items() to print each key-value pair.
11. Print the number of keys using len().
12. Stop.

Program

```
# Program demonstrating dictionary creation and manipulation
student = {"name": "Ravi", "age": 21, "department": "CSE"}
print("Dictionary:", student)

# Accessing values
```

```
print("Name:", student["name"])
print("Age using get():", student.get("age"))

# Adding a new key-value pair
student["cgpa"] = 8.5
print("After adding cgpa:", student)

# Modifying an existing value
student["age"] = 22
print("After modifying age:", student)

# Removing a key-value pair
student.pop("cgpa")
print("After removing cgpa:", student)

# Checking if a key exists
print("Is 'name' a key:", "name" in student)
print("Is 'marks' a key:", "marks" in student)

# Getting keys, values, and items
print("Keys:", student.keys())
print("Values:", student.values())
print("Items:", student.items())

# Looping through a dictionary
print("Dictionary contents:")
for key, value in student.items():
    print(f"{key}: {value}")

# Length of dictionary
print("Number of keys:", len(student))
```

Expected Output

```
Dictionary: {'name': 'Ravi', 'age': 21, 'department': 'CSE'}
Name: Ravi
Age using get(): 21
After adding cgpa: {'name': 'Ravi', 'age': 21, 'department': 'CSE',
'cgpa': 8.5}
After modifying age: {'name': 'Ravi', 'age': 22, 'department': 'CSE',
'cgpa': 8.5}
After removing cgpa: {'name': 'Ravi', 'age': 22, 'department': 'CSE'}
```

```
Is 'name' a key: True
Is 'marks' a key: False
Keys: dict_keys(['name', 'age', 'department'])
Values: dict_values(['Ravi', 22, 'CSE'])
Items: dict_items([('name', 'Ravi'), ('age', 22), ('department', 'CSE')])
Dictionary contents:
name: Ravi
age: 22
department: CSE
Number of keys: 3
```

Observation

- Program executed successfully with no errors.
- Values were correctly accessed using both indexing and get().
- A new key-value pair was successfully added, and an existing value was successfully modified.
- pop() correctly removed the specified key-value pair.
- The in operator correctly identified "name" as an existing key and "marks" as a non-existing key.
- keys(), values(), and items() correctly returned the dictionary's keys, values, and key-value pairs.
- The for loop correctly iterated through and printed each key-value pair.

Result

The objective of the experiment was achieved successfully. The program correctly demonstrated dictionary creation and manipulation, including accessing, adding, modifying, removing, key existence checking, and looping through key-value pairs.

Learning Outcome

- Learned how to create a dictionary and access its values using keys and get().
- Understood how to add, modify, and remove key-value pairs in a dictionary.
- Learned to use the in operator to check key existence.
- Understood the use of keys(), values(), and items() for retrieving dictionary contents.
- Learned how to loop through a dictionary using items() to access both keys and values together.

Experiment No. 7

Title

Object creation and usage.

Aim

To write a Python program to demonstrate the creation and usage of objects using a user-defined class, including constructors, instance variables, and methods.

Theory

- A class is a blueprint used to create objects in Python. An object is an instance of a class that contains its own data and can use the methods defined inside the class.
- The __init__() method is a constructor that is automatically called when an object is created. The self keyword refers to the current object. Instance variables store data belonging to a particular object, while methods define the operations that can be performed on the object.

Algorithm

1. Start.
2. Define a class named Student.
3. Create a constructor __init__() to initialize name, roll number, and marks.
4. Define a method display() to display student details.
5. Define a method calculate_grade() to determine the grade based on marks.
6. Create two objects of the Student class.
7. Call the methods using both objects.
8. Display the student details and grades.
9. Stop.

Program

```
# Program demonstrating object creation and usage

class Student:
    def __init__(self, name, roll_no, marks):
        self.name = name
        self.roll_no = roll_no
        self.marks = marks
```

```
    def display(self):
        print("Name:", self.name)
        print("Roll Number:", self.roll_no)
        print("Marks:", self.marks)

    def calculate_grade(self):
        if self.marks >= 90:
            return "A"
        elif self.marks >= 75:
            return "B"
        elif self.marks >= 60:
            return "C"
        elif self.marks >= 50:
            return "D"
        else:
            return "F"


# Creating objects
student1 = Student("Arun", 101, 82)
student2 = Student("Meena", 102, 68)

print("Student 1")
student1.display()
print("Grade:", student1.calculate_grade())

print("\nStudent 2")
student2.display()
print("Grade:", student2.calculate_grade())
```

Expected Output

Student 1
Name: Arun
Roll Number: 101
Marks: 82
Grade: B

Student 2
Name: Meena
Roll Number: 102
Marks: 68
Grade: C

Observation

- The Student class was successfully created.
- Two objects were created from the same class.
- Each object stored its own name, roll number, and marks.
- The display() method displayed the object details.
- The calculate_grade() method returned the appropriate grade.

Result

The objective of the experiment was achieved successfully. The program demonstrated class creation, object creation, constructors, instance variables, and methods.

Learning Outcome

- Learned how to define a class in Python.
- Understood how objects are created from a class.
- Learned the purpose of the __init__() constructor.
- Understood the use of self.
- Learned how objects can call methods to perform operations.

Experiment No. 8

Title

Program involving inheritance.

Aim

To write a Python program demonstrating inheritance by creating a child class that inherits properties and methods from a parent class.

Theory

- Inheritance is an object-oriented programming concept in which one class acquires the properties and methods of another class. The existing class is called the **parent class** or **base class**, while the new class is called the **child class** or **derived class**.

- Inheritance promotes code reusability because common features can be defined in the parent class and reused by child classes. A child class can also define its own additional methods.

Algorithm

1. Start.
2. Define a parent class Employee.
3. Create a constructor to initialize employee name and salary.
4. Define a method to display employee details.
5. Define a child class Developer that inherits from Employee.
6. Add an additional attribute called language.
7. Define a method to display the programming language.
8. Create an object of the Developer class.
9. Call the inherited and child-class methods.
10. Stop.

Program

```
# Program demonstrating inheritance

class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def display_employee(self):
```

```
        print("Employee Name:", self.name)
        print("Salary:", self.salary)


class Developer(Employee):
    def __init__(self, name, salary, language):
        super().__init__(name, salary)
        self.language = language

    def display_skill(self):
        print("Programming Language:", self.language)


# Creating object of child class
developer1 = Developer("Ravi", 55000, "Python")

print("Developer Details")
developer1.display_employee()
developer1.display_skill()
```

Expected Output

Developer Details
Employee Name: Ravi
Salary: 55000
Programming Language: Python

Observation

- Employee was created as the parent class.
- Developer inherited the properties and method of Employee.
- The super() function was used to call the parent class constructor.
- The child class also contained its own language attribute and display_skill() method.

Result

The objective of the experiment was achieved successfully. The program demonstrated inheritance, constructor reuse using super(), and the addition of child-class features.

Learning Outcome

- Learned the concept of inheritance.
- Understood parent and child classes.

- Learned how a child class can reuse parent-class methods.
- Understood the use of super().
- Learned how a child class can contain additional properties and methods.

Experiment No. 9

Title

Program involving overloading.

Aim

To write a Python program demonstrating operator overloading using the __add__() special method.

Theory

- Operator overloading allows operators such as +, -, *, and == to perform operations on user-defined objects.
- Python provides special methods, also called **dunder methods**, for operator overloading. For example, __add__() defines how the + operator behaves when it is used with objects of a class.

Algorithm

1. Start.
2. Define a class named Distance.
3. Create a constructor to initialize meters.
4. Define the __add__() method.
5. Add the meters of two Distance objects.
6. Return a new Distance object containing the result.
7. Create two distance objects.
8. Add the objects using the + operator.
9. Display the result.
10. Stop.

Program

```
# Program demonstrating operator overloading

class Distance:
    def __init__(self, meters):
        self.meters = meters

    def __add__(self, other):
        return Distance(self.meters + other.meters)
```

```
    def display(self):
        print("Distance:", self.meters, "meters")


# Creating objects
d1 = Distance(250)
d2 = Distance(175)

print("First Distance")
d1.display()

print("Second Distance")
d2.display()

# Operator overloading
total = d1 + d2

print("Total Distance")
total.display()
```

Expected Output

First Distance
Distance: 250 meters
Second Distance
Distance: 175 meters
Total Distance
Distance: 425 meters

Observation

- Two objects of the Distance class were created.
- The __add__() method defined the behavior of the + operator.
- The expression d1 + d2 called the overloaded __add__() method.
- A new object containing the total distance was returned.

Result

The objective of the experiment was achieved successfully. The program demonstrated operator overloading using the __add__() special method.

Learning Outcome

- Learned the concept of operator overloading.
- Understood the purpose of special methods.
- Learned how __add__() can customize the + operator.
- Understood how operators can be used with user-defined objects.

Experiment No. 10

Title

Reading and writing text and binary files.

Aim

To write a Python program for reading and writing data to text and binary files.

Theory

- Files are used to store data permanently. Python provides the open() function for performing file operations.
- A **text file** stores data in a human-readable form and is generally opened using modes such as r and w. A **binary file** stores data in the form of bytes and is opened using modes such as rb and wb.
- The with statement is commonly used for file handling because it automatically closes the file after the operation is completed.

Algorithm

10. Start.
11. Create a text file using write mode.
12. Write student information into the text file.
13. Open the text file in read mode.
14. Read and display its contents.
15. Create a binary file using binary write mode.
16. Convert a list of marks into bytes.
17. Write the bytes into the binary file.
18. Read the binary file using binary read mode.
19. Convert the data back into a list.
20. Display the binary data.
21. Stop.

Program

```
# Program for reading and writing text and binary files

import pickle

# Writing to a text file
```

```
with open("student.txt", "w") as file:
    file.write("Name: Arun\n")
    file.write("Department: Data Science\n")
    file.write("Marks: 85\n")

# Reading from the text file
with open("student.txt", "r") as file:
    text_data = file.read()

print("Text File Content:")
print(text_data)

# Writing to a binary file
marks = [85, 78, 92, 88]

with open("marks.dat", "wb") as file:
    pickle.dump(marks, file)

# Reading from the binary file
with open("marks.dat", "rb") as file:
    binary_data = pickle.load(file)

print("Binary File Data:")
print(binary_data)
```

Expected Output

Text File Content:
Name: Arun
Department: Data Science
Marks: 85

Binary File Data:
[85, 78, 92, 88]

Observation

- The text file was successfully created and written using `w` mode.
- The contents were read using `r` mode.
- The marks list was stored in a binary file using `pickle`.
- The binary data was successfully read and converted back into a Python list.

Result

The objective of the experiment was achieved successfully. The program demonstrated reading and writing operations for both text and binary files.

Learning Outcome

- Learned how to create and access files in Python.
- Understood text and binary file differences.
- Learned file modes such as r, w, rb, and wb.
- Understood the use of with for safe file handling.
- Learned how pickle can store Python objects in binary form.

Experiment No. 11

Title

Combining and merging datasets.

Aim

To write a Python program using Pandas to combine datasets using concatenation and merge related datasets using a common column.

Theory

- Pandas provides several methods for combining datasets.
- The concat() function is used to combine DataFrames along rows or columns. It is useful when datasets have a similar structure.
- The merge() function combines two DataFrames using one or more common columns, similar to a database join. Common types include inner, left, right, and outer joins.

Algorithm

11. Start.
12. Import the Pandas library.
13. Create two DataFrames containing employee records.
14. Combine the DataFrames using `concat()`.
15. Create another DataFrame containing department information.
16. Merge the employee dataset with the department dataset using `dept_id`.
17. Display the combined and merged datasets.
18. Stop.

Program

```
# Program demonstrating combining and merging datasets

import pandas as pd

# First dataset
employees1 = pd.DataFrame({
    "emp_id": [101, 102, 103],
    "name": ["Arun", "Meena", "Ravi"],
    "dept_id": [1, 2, 1]
})
```

```
# Second dataset
employees2 = pd.DataFrame({
    "emp_id": [104, 105],
    "name": ["Priya", "Kiran"],
    "dept_id": [2, 3]
})

# Combining datasets
combined = pd.concat([employees1, employees2], ignore_index=True)

print("Combined Employee Dataset:")
print(combined)

# Department dataset
departments = pd.DataFrame({
    "dept_id": [1, 2, 3],
    "department": ["Data Science", "Computer Science", "Mathematics"]
})

# Merging datasets
merged = pd.merge(combined, departments, on="dept_id", how="inner")

print("\nMerged Dataset:")
print(merged)
```

Expected Output

Combined Employee Dataset:

	emp_id	name	dept_id
0	101	Arun	1
1	102	Meena	2
2	103	Ravi	1
3	104	Priya	2
4	105	Kiran	3

Merged Dataset:

	emp_id	name	dept_id	department
0	101	Arun	1	Data Science
1	102	Meena	2	Computer Science
2	103	Ravi	1	Data Science
3	104	Priya	2	Computer Science
4	105	Kiran	3	Mathematics

Observation

- Two employee datasets were combined using pd.concat().
- The resulting DataFrame contained records from both datasets.
- The department dataset was merged using the common dept_id column.
- The inner merge returned records having matching department IDs.

Result

The objective of the experiment was achieved successfully. The program demonstrated combining datasets using concat() and merging related datasets using merge().

Learning Outcome

- Learned how to combine DataFrames using concat().
- Understood how datasets can be merged using a common column.
- Learned the purpose of the on parameter.
- Understood the basic concept of database-style joins in Pandas.

Experiment No. 12

Title

Program involving regular expressions.

Aim

To write a Python program using regular expressions to extract and validate information from a text string.

Theory

- Regular expressions, commonly called **regex**, are patterns used for searching, matching, and extracting text.
- Python provides the re module for working with regular expressions. Functions such as search(), findall(), and match() can be used for different pattern-matching operations.
- In this program, regular expressions are used to extract email addresses and phone numbers and to validate an email address.

Algorithm

22. Start.
23. Import the re module.
24. Create a string containing names, email addresses, and phone numbers.
25. Define a regular expression pattern for email addresses.
26. Use re.findall() to extract all email addresses.
27. Define a pattern for 10-digit phone numbers.
28. Extract phone numbers from the text.
29. Define a sample email address for validation.
30. Use re.match() to check whether the email follows the required pattern.
31. Display the extracted and validated information.
32. Stop.

Program

```
# Program demonstrating regular expressions

import re

text = """
Contact Angel at angel@gmail.com or call 9876543210.
You can also contact Merlin at merlin.ds@yahoo.com
or call 9123456780.
```

```
"""

# Pattern for email addresses
email_pattern = r'[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}'

emails = re.findall(email_pattern, text)

# Pattern for 10-digit phone numbers
phone_pattern = r'\b\d{10}\b'

phone_numbers = re.findall(phone_pattern, text)

print("Email Addresses:")
for email in emails:
    print(email)

print("\nPhone Numbers:")
for phone in phone_numbers:
    print(phone)

# Email validation
test_email = "student@example.com"

if re.match(email_pattern, test_email):
    print("\nValid Email:", test_email)
else:
    print("\nInvalid Email:", test_email)
```

Expected Output

Email Addresses:
angel@gmail.com
merlin.ds@yahoo.com

Phone Numbers:
9876543210
9123456780

Valid Email: student@example.com

Observation

- The re module was successfully used for pattern matching.
- re.findall() extracted all matching email addresses and phone numbers.
- The email pattern checked the structure of an email address.
- The phone pattern identified 10-digit numbers.
- The test email matched the specified pattern.

Result

The objective of the experiment was achieved successfully. The program demonstrated the use of regular expressions for searching, extracting, and validating text patterns.

Learning Outcome

- Learned the purpose of regular expressions.
- Understood the use of the re module.
- Learned to use findall() for extracting multiple matches.
- Learned how patterns can be used for email and phone-number validation.
- Understood basic regex symbols such as \d, \b, and character classes.

Experiment No. 13

Title

Data aggregation and group-wise operations.

Aim

To write a Python program using Pandas to perform data aggregation and group-wise operations on a dataset.

Theory

- Data aggregation is the process of summarizing data using functions such as sum(), mean(), count(), min(), and max().
- Pandas provides the groupby() function to divide data into groups based on one or more columns. Aggregation functions can then be applied to each group to obtain useful summaries.
- For example, employee salary data can be grouped according to department to calculate the number of employees, average salary, and total salary for each department.

Algorithm

1. Start.
2. Import the Pandas library.
3. Create a DataFrame containing employee names, departments, and salaries.
4. Group the data using the department column.
5. Calculate the number of employees in each department.
6. Calculate the average salary for each department.
7. Calculate the total salary for each department.
8. Calculate the maximum salary for each department.
9. Display the aggregated results.
10. Stop.

Program

```
# Program demonstrating data aggregation and group-wise operations

import pandas as pd

data = {
    "Name": ["Aila", "Mira", "Rosy", "Prem", "Rathna", "Diya"],
    "Department": [
        "Data Science",
```

```
        "Data Science",
        "Computer Science",
        "Computer Science",
        "Data Science",
        "Computer Science"
    ],
    "Salary": [55000, 60000, 50000, 65000, 58000, 62000]
}

df = pd.DataFrame(data)

print("Employee Dataset:")
print(df)

# Group-wise aggregation
summary = df.groupby("Department").agg(
    Employee_Count=("Name", "count"),
    Average_Salary=("Salary", "mean"),
    Total_Salary=("Salary", "sum"),
    Maximum_Salary=("Salary", "max")
)

print("\nDepartment-wise Summary:")
print(summary)
```

Expected Output

Employee Dataset:

	Name	Department	Salary
0	Aila	Data Science	55000
1	Mira	Data Science	60000
2	Rosy	Computer Science	50000
3	Prem	Computer Science	65000
4	Rathna	Data Science	58000
5	Diya	Computer Science	62000

Department-wise Summary:

Department	Employee_Count	Average_Salary	Total_Salary	Maximum_Salary
Computer Science	3	59000.000000	177000	65000
Data Science	3	57666.666667	173000	60000

Observation

- The employee data was successfully stored in a Pandas DataFrame.
- groupby() divided the employees according to their departments.
- count() calculated the number of employees in each department.
- mean() calculated the average salary.
- sum() calculated the total salary.
- max() identified the highest salary in each department.
    -

Result

The objective of the experiment was achieved successfully. The program demonstrated data aggregation and group-wise operations using Pandas.

Learning Outcome

- Learned the concept of data aggregation.
- Understood how groupby() divides data into groups.
- Learned to apply multiple aggregation functions.
- Understood the use of count(), mean(), sum(), and max().
- Learned how grouped summaries can be used for data analysis.